Device-Independent I/O Software

1 Basic Idea

Definition

Device-independent I/O software is the part of the operating system that performs I/O functions common to many devices and presents a uniform interface to upper layers.

- Some I/O code must be device-specific and belongs in device drivers.
- Other I/O work is common across devices and belongs in device-independent OS software.
- The exact boundary between drivers and device-independent software depends on the system.
- The main purpose is to make different devices look similar to the rest of the OS.

2 Main Functions

Function	Meaning
Uniform interface	Makes drivers of the same class support the same OS-visible operations.
Buffering	Temporarily stores input or output data to handle timing, paging, and rate differences.
Error reporting	Provides a common framework for reporting and escalating I/O errors.
Dedicated devices	Controls exclusive-use devices such as printers.
Logical block size	Hides physical sector-size differences from higher layers.

Core Point

Device-independent I/O software hides differences that higher layers should not need to know.

3 Uniform Driver Interfaces

Problem

If every driver has a different interface, the operating system must be modified whenever a new device or driver is added.

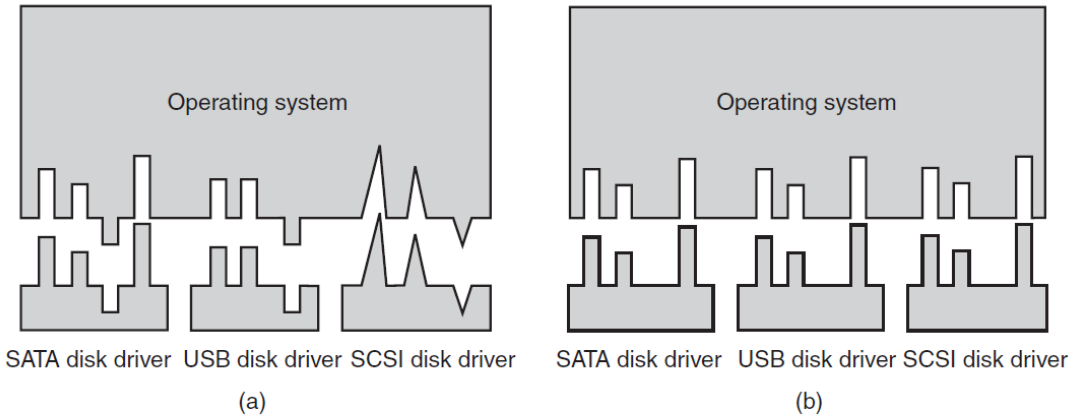

Figure 5-14. (a) Without a standard driver interface. (b) With a standard driver interface.

With a standard interface, all drivers in a class obey the same contract. Driver writers know what procedures they must provide, and the OS can call drivers uniformly through known entry points.

Driver Interface Table

For each device class, the OS can define a table of function pointers that every driver in that class must provide.

Device class	Typical standard operations
Disk class	Read block, write block, format, power control, status check.
Printer class	Write character stream, check status, start output, stop output.
Network class	Send packet, receive packet, set address, report link status.

4 Device Naming and Protection

Uniform Naming

Device-independent software maps symbolic device names to the correct driver and device unit.

In UNIX-like systems, devices appear as special files in the file system. A name such as `/dev/disk0` identifies a special file whose i-node contains device-identifying information.

Major and Minor Numbers

Major number = which driver. Minor number = which device controlled by that driver.

Device access must also be protected. If devices are named objects, normal file-protection rules can apply, and the system administrator can set permissions for each device.

5 Buffering

Definition

Buffering means using temporary memory storage to hold I/O data before it reaches its final source or destination.

- Buffering is needed for both block devices and character devices.
- It handles mismatches between device speed and process speed.
- It can avoid waking a user process for every small piece of data.
- It can solve problems caused by user pages being paged out.
- It can improve output behavior by letting the user reuse a buffer quickly.

6 Input Buffering Methods

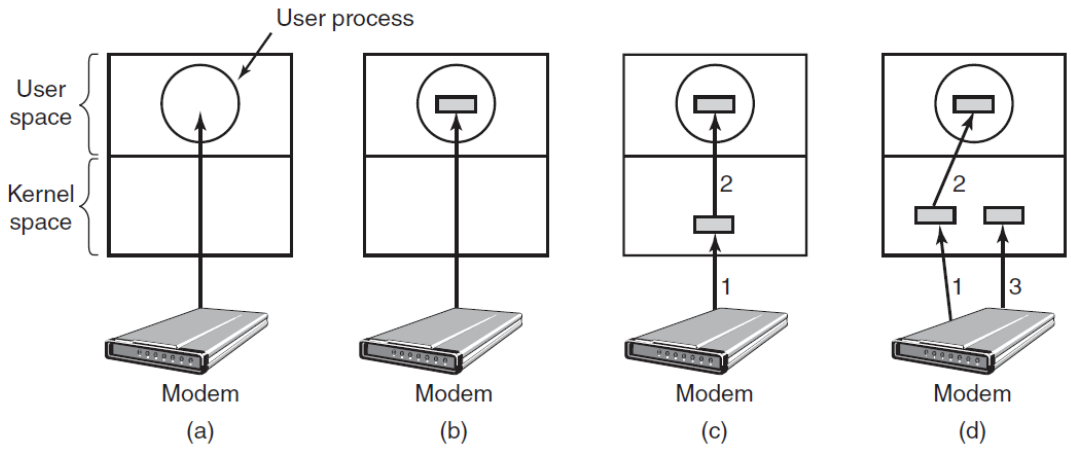

Figure 5-15. (a) Unbuffered input. (b) Buffering in user space. (c) Buffering in the kernel followed by copying to user space. (d) Double buffering in the kernel.

Method	Meaning and drawback
Unbuffered input	The user process waits for one character at a time and may be restarted for every character.
User-space buffer	Reduces wakeups, but the buffer page may be paged out when input arrives.
Kernel buffer	The interrupt handler stores characters in a kernel buffer; data are copied to user space later.
Double buffering	Two kernel buffers alternate, so one can be copied while the other receives new input.

Double Buffering

Double buffering avoids losing input while one full kernel buffer is being copied to user space.

7 Circular Buffers and Output Buffering

Circular Buffer

A **circular buffer** is a fixed memory region with two moving pointers: one for inserting new data and one for removing old data.

Circular buffers are common in continuous data streams such as networking. Hardware or the driver advances the input pointer as data arrive, while the OS advances the output pointer as data are consumed. Both pointers wrap around at the end.

For output, the OS can copy a user's data into a kernel buffer and unblock the process immediately. The actual device output continues later from the kernel buffer.

Buffer Ownership

Copying output to a kernel buffer makes it safe for the user process to reuse its original buffer immediately.

8 Cost of Too Much Buffering

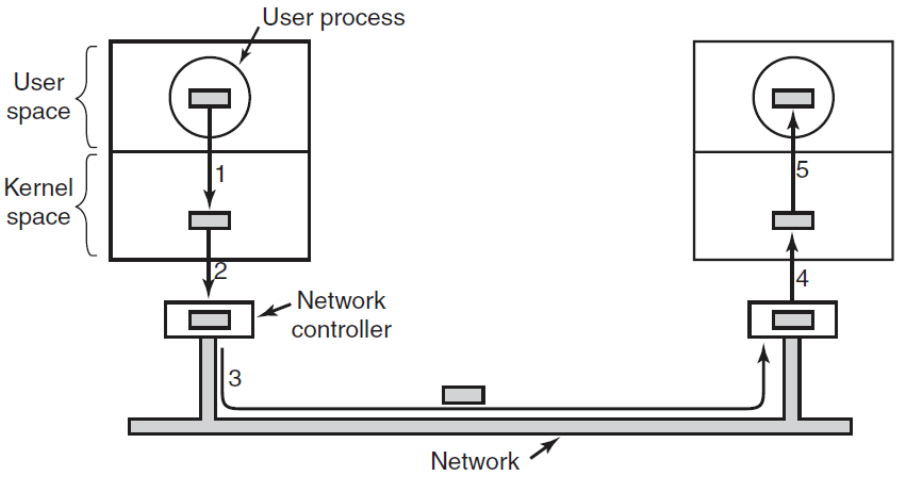

Figure 5-16. Networking may involve many copies of a packet.

1. Sender copies the packet from user space to a kernel buffer.
2. The driver copies the packet to the network controller.
3. The network controller sends the packet across the network.
4. The receiver copies the packet from its controller to a kernel buffer.
5. The receiver copies the packet from the kernel buffer to the receiving process's buffer.

Performance Cost

Buffering is useful, but repeated copying can slow down I/O because the copies happen sequentially.

9 Error Reporting

Goal

Device-independent software provides a common error-reporting framework, even though many I/O errors are handled inside specific drivers.

Error type	Handling approach
Programming error	Return an error code to the caller.
Invalid operation	Examples include writing to an input device or reading from an output device.
Invalid parameter	Examples include an invalid buffer address or invalid device number.
Actual I/O error	The driver tries recovery first; if it cannot recover, the problem is passed upward.
Critical system damage	The system may have to print an error message and terminate.

10 Dedicated Device Allocation and Block Size

Dedicated Device

A **dedicated device** can be used by only one process at a time.

Printers are a common example. The OS may fail an open request if the device is unavailable, or it may block the caller in a queue until the device becomes available.

Logical Block Size

Device-independent software can present a uniform logical block size even when physical devices use different sector sizes.

The OS can treat several physical sectors as one logical block. Higher layers then work with abstract devices that share one block size, even when the physical hardware differs.

Final Point

Device-independent I/O software standardizes interfaces, names, protection, buffering, error handling, dedicated-device allocation, and logical block sizes across different devices.